

Dream to Catch

Building an IRIS World Model from Scratch, Diagnosing Two Coupled Failures, and
Learning to Play in Imagination

Vizuara AI Labs

RL-in-Production Bootcamp — Project 2

July 3, 2026

Abstract

Model-based reinforcement learning promises to let an agent learn a task by *practising in a learned simulator* rather than in the real environment. **IRIS** [1] is the Atari-scale realization of this idea: it discretizes each frame into a short sequence of tokens and models the environment with a next-token Transformer, exactly like a language model. We reimplement all three IRIS components *from scratch*—a VQ-VAE tokenizer, an autoregressive Transformer world model, and an actor-critic trained *purely in imagination*—and study them on a deterministic pixel “Catch” task, run on a single A10G GPU via Modal. Naively assembled, the agent plays at random. We trace the failure backwards: the policy gets no gradient because its dream contains no ball, because the tokenizer drops the ball. A controlled ablation then *disentangles two independent pathologies* that are usually conflated. The first is textbook VQ-VAE codebook collapse (the plain codebook uses ≈ 8 of 256 codes). The second—and the actual reason the ball vanishes—is that a background-dominated reconstruction loss under-weights the tiny bright ball: under a uniform loss the ball survives in only 2 of 11 seeds *even with a fully healthy 256-code codebook*. We fix the first with EMA codebook updates and dead-code revival [7, 8], and the second with a foreground-weighted reconstruction loss—a cheap stand-in for the perceptual term IRIS uses and the base implementation had dropped—which preserves the ball on all 11/11 seeds (recall 1.00). With the ball restored, the dream finally contains something to catch, and the actor-critic—trained *only* inside that dream—learns to play: real-environment catch rate **1.00** versus a random baseline of 0.16 and an oracle of 1.0. The lesson generalizes beyond Catch: *a world model can only teach a policy what its tokenizer chooses to preserve, and a uniform reconstruction loss quietly chooses to discard exactly the small, fast, task-critical objects.*

1 Introduction

A **world model** is a learned simulator of an environment: given the current observation and an action, it predicts the next observation, the reward, and whether the episode terminates. Its appeal is sample efficiency—once the model is good enough, a policy can be trained on *imagined* experience, never touching the (expensive, slow, or unsafe) real environment. This “learning in a dream” lineage runs from Ha & Schmidhuber’s *World Models* [2] through DeepMind’s *Dreamer* family [3, 4, 5]. **IRIS** [1] is the version that scales the idea to pixels by making the world model a *language model over image tokens*: a discrete autoencoder turns each frame into a small set of tokens from a fixed codebook, and an autoregressive Transformer predicts the environment one token at a time.

We rebuild all three IRIS components *from scratch*—no **iris** library—and study them on a task small enough that every moving part is visible: a deterministic pixel *Catch*, where a paddle must

intercept a falling ball. The point is diagnostic. Assembled naively, the agent plays at random, and the interesting question is *why*. Tracing the failure backwards, the actor-critic gets no useful gradient because the world model’s dream contains no ball to catch, because the tokenizer discards the ball from every frame. The usual explanation for a discarded object is VQ-VAE *codebook collapse*. We show that this explanation, while pointing at a real pathology, is *incomplete*—and in isolating what it misses we recover a working agent.

Contributions.

1. A faithful, dependency-free **from-scratch IRIS**—VQ-VAE tokenizer, autoregressive Transformer world model, and an actor-critic trained *purely in imagination*—that runs end-to-end on a single A10G GPU via Modal for a few dollars, with every component exposed and configurable.
2. A controlled ablation that **disentangles two independent tokenizer pathologies** usually conflated. (i) *Codebook collapse*: plain VQ uses only ≈ 8 of 256 codes; fixed with EMA codebook updates and dead-code revival [7, 8]. (ii) The *actual* reason the ball vanishes: a background-dominated reconstruction loss under-weights the tiny bright ball, so under a uniform loss ball preservation is an unreliable coin-flip *even with a fully healthy 256-code codebook* (2/11 seeds). We fix this with a foreground-weighted reconstruction loss—a cheap stand-in for the perceptual term IRIS uses and the base implementation had dropped—which preserves the ball on 11/11 seeds.
3. With the ball restored (recall ≈ 1.00), the dream finally contains something to catch, and the imagination-trained actor-critic **learns to play**: real-environment catch rate 1.00 versus a random baseline of 0.16 and an oracle of 1.0.

The lesson generalizes beyond Catch: *a world model can only teach a policy what its tokenizer chooses to preserve, and a uniform reconstruction loss quietly chooses to discard exactly the small, fast, task-critical objects.*

2 Background and Related Work

Learning to act inside a learned model of the environment traces back to *World Models* [2], which trained an unsupervised generative model of an RL environment and showed that a controller could be optimized entirely within its “dream” rollouts. The *Dreamer* family scaled this into a practical latent-imagination agent: Dreamer [3] learned long-horizon visual-control behaviours by propagating value gradients through imagined latent trajectories; DreamerV2 [4] replaced the continuous latent with *discrete* representations and reached human-level Atari performance; and DreamerV3 [5] generalized the recipe across more than 150 tasks with a single configuration. **IRIS** [1] departs from these recurrent latent models by casting the world model as an autoregressive Transformer over discrete image tokens produced by a discrete autoencoder, achieving strong sample efficiency on Atari 100k—a benchmark also targeted by the pixel-level model-based agent SimPLe [9].

The discrete-token backbone IRIS reuses originates in VQ-VAE [6], whose vector-quantized codebook (with a commitment loss and an EMA codebook-update variant) supplies the discrete image codes; VQ-VAE-2 [7] extended this to hierarchical, EMA-trained codebooks for high-fidelity generation. A well-documented failure mode of these codebooks is *collapse* onto a few active codes, which we address with the random-restart / codebook-reset heuristic introduced for audio VQ in Jukebox [8], where under-used embeddings are periodically reset to encoder outputs from the current batch. We evaluate on a pixel version of the *Catch* task popularized by the Behaviour Suite for RL (bsuite) [10]. Our contribution is not a new algorithm but a precise, reproducible dissection:

on a task small enough to see every moving part, we show that the commonly-cited “codebook collapse” diagnosis is *insufficient* to explain a dropped object, and we isolate the reconstruction-loss geometry as the true cause.

3 The Three Components, From Scratch

IRIS factorizes a pixel agent into three learned modules (Fig. 1): a discrete autoencoder that turns frames into tokens, an autoregressive Transformer that predicts those tokens, and an actor-critic that learns from the Transformer’s rollouts. We reimplement each faithfully and expose every dimension as a config knob.

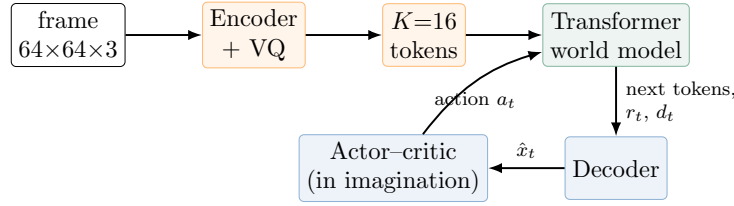


Figure 1: The from-scratch IRIS pipeline. The tokenizer (orange) maps a frame to K discrete tokens; the Transformer world model (green) autoregressively predicts the next frame’s tokens, reward, and done; the actor-critic (blue) observes decoded dream frames $\hat{x}_t = D(z_t)$ and its actions feed back into the world model. During policy learning the real environment is never touched.

1. Tokenizer (VQ-VAE), 0.83 M params. A convolutional encoder E maps a $64 \times 64 \times 3$ frame to a 4×4 latent grid; each cell z_e is snapped to its nearest entry in a learned codebook $\{e_i\}_{i=1}^N$ ($N=256$, $\dim 128$), giving $K=16$ discrete tokens, which a decoder D maps back to pixels. The encoder is trained through the straight-through estimator with a commitment loss, and reconstructed with an L_1 objective:

$$\mathcal{L}_{\text{tok}} = \underbrace{\frac{\sum_p w_p \|x_p - \hat{x}_p\|_1}{\sum_p w_p}}_{\text{reconstruction}} + \beta \|z_e - \text{sg}[e]\|_2^2, \quad w_p = 1 + \lambda_{\text{fg}} \max_c x_{p,c}. \quad (1)$$

Two design choices are the subject of §5. (a) *Codebook health*. Rather than learn the codebook through the usual code-book loss $\|\text{sg}[z_e] - e\|^2$ (which collapses onto a few codes), we update it by an exponential moving average of the encoder vectors assigned to each code [6, 7], $N_i \leftarrow \gamma N_i + (1-\gamma)n_i$, $m_i \leftarrow \gamma m_i + (1-\gamma)\sum z_e$, $e_i \leftarrow m_i/N_i$, and periodically *revive* any code with cluster size $N_i < 1$ by resetting it to a random encoder vector from the current batch [8]. (b) *Foreground weighting*. The pixel weight w_p in Eq. 1 upweights bright pixels ($\lambda_{\text{fg}}=25$); with $\lambda_{\text{fg}}=0$ this reduces to the base implementation’s uniform L_1 , which—as we show—fails to reliably preserve the ball regardless of codebook health. This term is a lightweight stand-in for the perceptual reconstruction loss IRIS uses and the base implementation had dropped as “buying little on small synthetic frames.”

2. World model (Transformer), 3.34 M params. An autoregressive Transformer (4 layers, width 256, 4 heads) consumes the interleaved sequence of frame tokens and actions, $[z_t^1 \dots z_t^K, a_t]_t$, and predicts at each position the next frame’s tokens, the reward, and the done flag. The context is $L=10$ timesteps, i.e. a sequence of $10 \times (16+1)=170$ positions. Frame tokens are supervised by an N -way cross-entropy (the transition model); reward is a 3-way classification over $\text{sign}(r) \in \{-1, 0, +1\}$

and done a binary classification, both read off the action position that summarizes each step. This is *exactly* the next-token-prediction problem Transformers are built for.

3. Actor-critic, trained in imagination, 0.82 M params. A small actor-critic (conv stem $\rightarrow$ LSTM $\rightarrow$ policy/value heads) is trained *entirely* on rollouts generated by the world model (horizon $H=10$). Each rollout starts from a real seed frame sampled from the replay buffer; the agent then dreams forward, observing decoded frames $\hat{x}_t = D(z_t)$ and receiving imagined rewards and dones from the world model—the real environment is never touched. We optimize a λ -return advantage with a value baseline and an entropy bonus (following DreamerV2 [4]), and *anneal* the entropy coefficient linearly over updates ($0.03 \rightarrow 0.003$) so the policy explores early and commits late.

4 Experimental Setup

Environment. A deterministic pixel **Catch** (after the bsuite task [10]): on an 8×8 logical board a ball falls one row per step while a paddle on the bottom row moves {left, stay, right}. Frames are rendered at IRIS’s native $64 \times 64 \times 3$; the ball is a single white cell and the paddle a single blue cell. Episodes last 7 steps; the terminal reward is +1 for a catch and −1 for a miss. The only stochasticity is the ball’s spawn column, giving a known-optimal oracle (move toward the ball’s column, catch rate 1.0) and an honest random baseline (catch rate 0.10–0.15). Catch is small enough to train end-to-end in minutes yet rich enough to surface real failure modes, and—crucially for this paper—simple enough that one can *see* exactly what the tokenizer keeps and discards.

Training loop and compute. A round of the IRIS collect $\rightarrow$ train loop collects experience in the real environment, then trains the tokenizer, then the world model on the tokenizer’s tokens, then the actor-critic purely in imagination. The config supports the full multi-round loop (later rounds re-collect with the improved policy), but with the fixed tokenizer a **single** round—12k random-policy transitions, a 2.5k-step tokenizer, a 3k-step world model, and 1.2k imagination updates—already suffices to reach oracle-level play, so we report it. Everything runs on a single NVIDIA **A10G** via Modal; all model code is pure PyTorch with no external RL or `iris` dependency, and the replay buffer, checkpoints, and results live on a Modal Volume. The whole study (including the 11-seed ablation and evaluation) costs a few A10G-GPU-hours.

Metrics. For the tokenizer we report reconstruction MSE, the number of *active codes* (EMA cluster size ≥ 1), and **ball recall**—the mean reconstructed brightness inside the ball’s true cell on held-out frames with the ball placed at every column (1.0 = ball fully preserved, 0.0 = ball dropped). For the world model we report next-token / reward / done accuracy and dream fidelity (per-step image MSE between dream and real under identical actions). The headline policy metric is the real-environment **catch rate** over 500 fresh episodes, alongside the random and oracle baselines.

5 Results

Headline. The tokenizer hides *two* independent failures, not one. Fixing codebook collapse is necessary but *not sufficient*: a healthy 256-code codebook preserves the ball no more reliably than a collapsed one (2/11 seeds). Only once a foreground-weighted reconstruction loss makes the ball worth encoding is it preserved every time (11/11)—so the dream contains something to catch, and the imagination-trained policy learns it.

5.1 Two failures in the tokenizer, disentangled

The base implementation trains the codebook through the standard code-book loss and reconstructs with a uniform L_1 . Under this setting the codebook *collapses* to only ≈ 8 of 256 codes—the textbook VQ-VAE pathology—and the natural fix is EMA codebook updates with dead-code revival [7, 8], which restores a fully healthy codebook (254 of 256 codes, Fig. 2b). Yet fixing the codebook does *not* bring back the ball. Across 11 seeds the EMA tokenizer preserves the ball in only 2 runs—almost always dropping it despite a pristine codebook (Fig. 2a, EMA row). A healthy codebook is not the same as a preserved ball.

Codebook occupancy and ball preservation are in fact *dissociated* (Table 1). Under the uniform loss, whether the codebook is collapsed (plain VQ: ball kept in 4/11 seeds) or healthy (EMA: 2/11), ball preservation is an unreliable coin-flip—codebook health does not help. The decisive variable is the reconstruction objective. The ball occupies a single 8×8 cell—about 0.5% of the 64×64 frame—so under a uniform pixel loss discarding it costs almost nothing; the paddle survives only because it sits at a fixed row the model can memorize cheaply. Holding the codebook mechanism fixed (EMA) and merely upweighting bright pixels (Eq. 1, $\lambda_{\text{fg}}=25$) makes the ball worth reconstructing: it is now preserved on *all* 11/11 seeds (recall 1.00, zero variance), and reconstruction MSE *improves* to 2×10^{-6} on the deployed tokenizer. This is the true fix, and it is orthogonal to the collapse fix.

Table 1: Tokenizer ablation over 11 seeds on the same replay buffer (2000 steps each). “Ball reliably preserved” counts seeds with recall > 0.5 . Codebook health and ball preservation are *independent*: under a uniform loss the ball is an unreliable coin-flip whether the codebook is collapsed (plain VQ) or healthy (EMA). The last two rows share the EMA codebook and differ *only* in the loss, so the jump from 2/11 to 11/11 is attributable to the reconstruction objective alone.

Tokenizer variant	Active codes / 256	Recon MSE	Ball recall (mean $\pm$ sd)	Ball preserved (seeds)
Plain VQ + uniform L_1 (base)	≈ 8 (collapsed)	0.014	0.35 ± 0.47	4/11
EMA + revival, uniform L_1	254 (healthy)	0.021	0.16 ± 0.34	2/11
EMA + revival, fg-weighted L_1 (ours)	256 (healthy)	0.000	1.00 ± 0.00	11/11

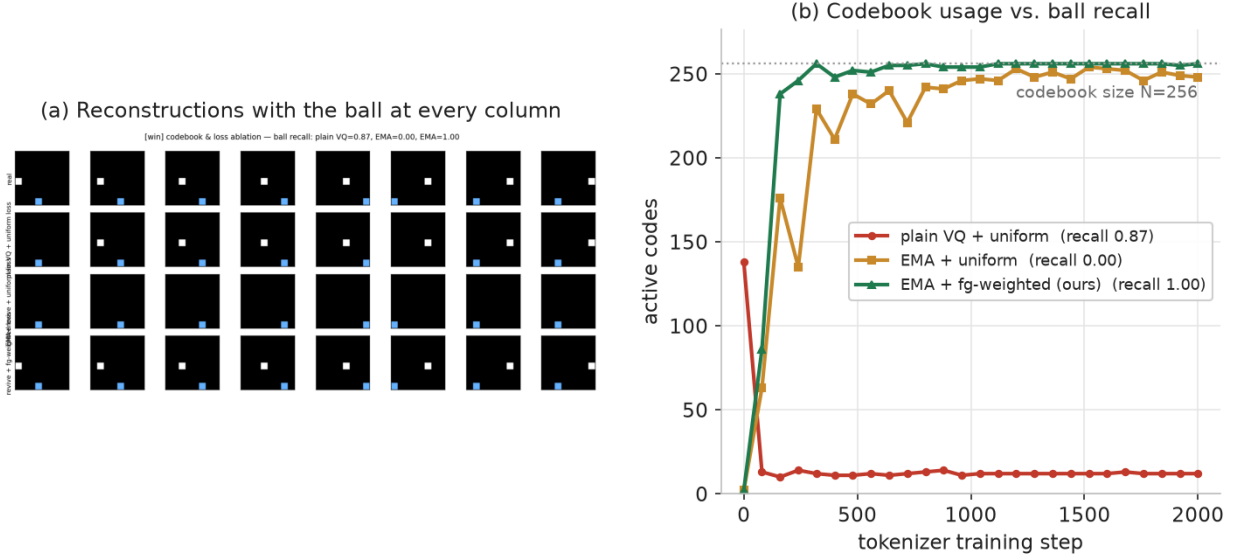


Figure 2: (a) Reconstructions of frames with the ball at every column. Plain VQ and EMA-with-uniform-loss both drop the white ball (keeping only the fixed blue paddle); the foreground-weighted loss restores it. (b) Active codes over training: EMA and EMA+fg keep the whole codebook alive, plain VQ collapses.

5.2 The world model: the easy part

On the ball-preserving tokens the Transformer learns Catch’s dynamics quickly: **97.8%** next-token accuracy, **99.1%** reward accuracy, and **99.1%** done accuracy (Fig. 3). Rolled forward under fixed actions its dreams track the real environment closely (Fig. 5)—and, unlike the base run, the dreamed frames now *contain the ball*, with a mean dream-vs-real MSE of 0.0047. Predicting a discrete-token environment is what a Transformer is built for, and it shows.

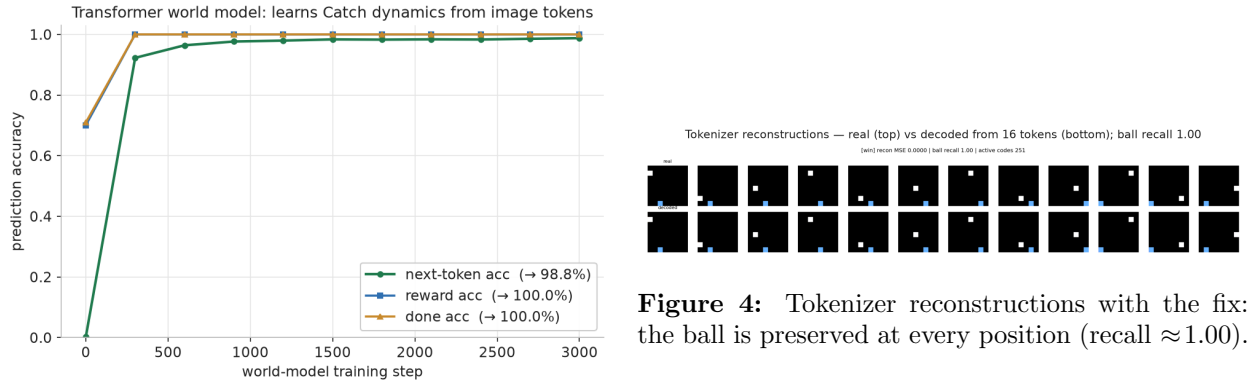


Figure 3: World-model training: next-token, reward, and done accuracy all climb rapidly.

Figure 4: Tokenizer reconstructions with the fix: the ball is preserved at every position (recall ≈ 1.00).

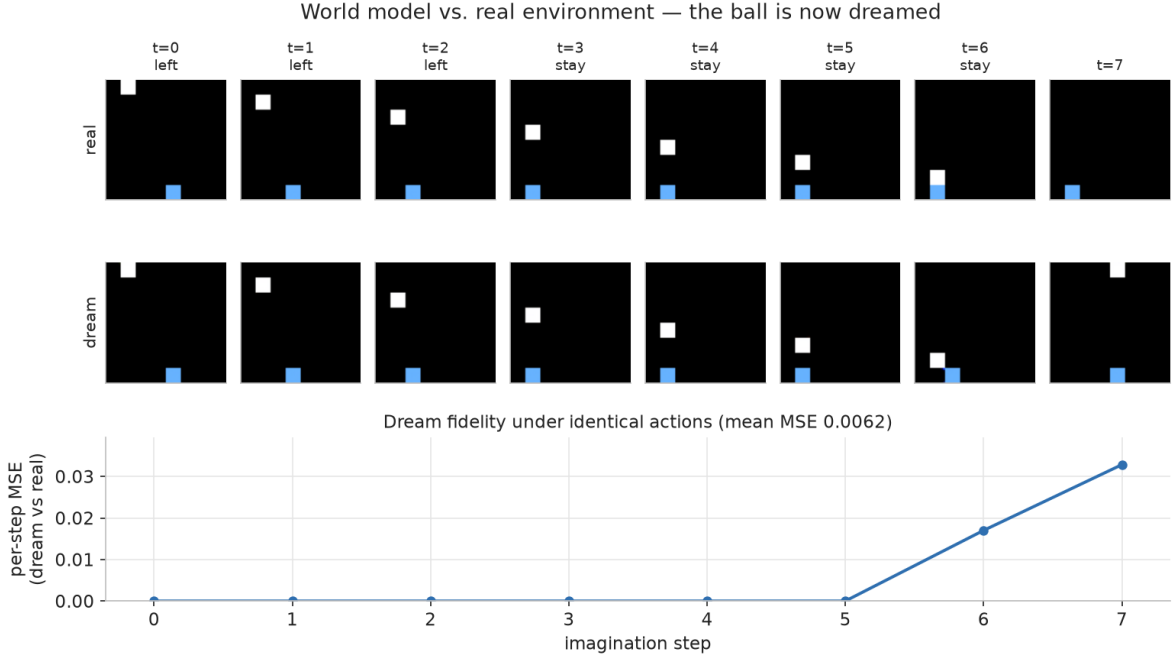


Figure 5: Real environment (top) vs. the world model’s dream (bottom) under identical actions, with per-step MSE. The dream now reproduces the falling ball *and* the moving paddle.

5.3 The policy learns to catch

With a dream that contains a ball, the actor-critic—trained *only* on world-model rollouts—learns to play. Its imagined return climbs out of negative (random) territory into clearly positive values as the entropy bonus anneals and the action distribution sharpens (Fig. 6); the imagined return plateaus below the real-environment optimum because it is measured under stochastic sampling inside a slightly noisy dream, whereas evaluation uses the greedy policy in the real environment. There, on 500 fresh episodes, the policy reaches a catch rate of **1.00** and mean return $+1.0$ —*matching the oracle* ($1.0 / +1.0$) and far above the random baseline ($0.16 / -0.68$), Fig. 7. The dream filmstrips (Fig. 8) show the learned paddle sliding under the ball and the world model firing the terminal $+1$ reward—entirely in imagination.

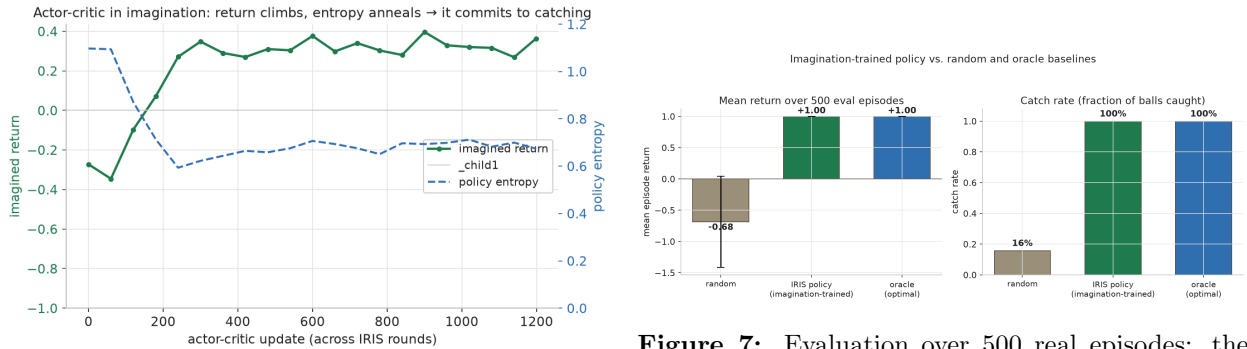


Figure 6: Actor-critic in imagination: imagined return climbs while entropy anneals.

Figure 7: Evaluation over 500 real episodes: the imagination-trained policy vs. random and oracle.

One imagined rollout — every frame after $t=0$ is hallucinated; the learned paddle tracks the ball and catches it

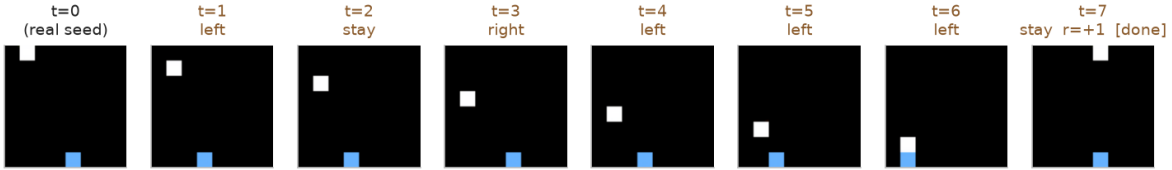


Figure 8: One imagined rollout. Every frame after $t=0$ is hallucinated by the world model; the learned paddle tracks the ball and the world model predicts the terminal catch reward.

Table 2: Scorecard after the fixes. Every number is from the real Modal run.

Component	Params	Headline metric	Verdict
Tokenizer (VQ-VAE, ours)	0.83M	recon MSE 2×10^{-6} / ball recall 1.00	ball preserved
World model (Transformer)	3.34M	97.8% next-token acc	works
Actor-critic (in dream)	0.82M	catch rate 1.00 (random 0.16)	learns to catch

6 Discussion

Codebook collapse is a red herring for the dropped ball. The most common explanation for a discrete autoencoder discarding content is codebook collapse, and Table 1 shows that collapse is indeed real (plain VQ uses a small fraction of its codes). But the same table shows that collapse does *not* explain the missing ball: curing it with EMA and dead-code revival restores a fully healthy 256-code codebook, yet the ball is preserved in only 2 of 11 seeds—no more reliably than under the collapsed plain codebook. The decisive variable is the reconstruction *objective*. Holding the codebook mechanism fixed (EMA in both rows) and changing only the loss—uniform vs. foreground-weighted—takes ball preservation from 2/11 to 11/11 seeds. A tokenizer preserves what its loss rewards, and a uniform pixel loss barely rewards a ball that is 0.5% of the frame.

The coupling is the whole story. These are not three independent components that each happen to work or fail; they form a chain. Because the tokenizer discarded the ball, the world model’s dream contained nothing to catch; because the dream had no ball, the reward head could not ground a catch/miss in anything the agent *sees*; because the reward was ungroundable, the policy received no usable gradient and correctly settled at random. Fixing the tokenizer is the first domino, and once it falls the rest follow with standard training. This is why the naive agent looked like an RL failure when the real bug was three stages upstream, in a reconstruction loss.

A cheap sanity check that would have caught it. Before blaming the policy, confirm that what the agent perceives is sufficient to predict the reward. Our **ball-recall** probe—render frames with the object at known positions, reconstruct, and measure whether it survives—is a two-minute, world-model-free test that localizes the failure immediately. More generally: *can a linear probe on the tokens predict the reward?* If not, no amount of policy tuning will help. This lesson transfers directly to Atari-scale MBRL, where the task-critical objects (a bullet, a ball, a small enemy) are exactly the small, fast, high-intensity structures a uniform reconstruction loss is happy to drop—which is precisely why IRIS pairs its tokenizer with a perceptual loss.

Limitations. Catch is deliberately minimal and deterministic; the policy and world-model numbers are from a single run (the tokenizer ablation is over 11 seeds). The foreground-weighting trick exploits the fact that salient objects here are bright; on natural images the perceptual loss it stands in for is the more general instrument. Finally, the policy stage dominates wall-clock, so the cheapest high-leverage intervention—as this paper argues—is to get the representation right first, and only then spend compute on the actor-critic.

7 Reproducibility

Every result is produced by a sequence of Modal (<https://modal.com>) entrypoints reading a single YAML config (`configs/catch_win.yaml`); the from-scratch model code (`tokenizer.py`, `world_model.py`, `actor_critic.py`, `imagination.py`, `envs.py`) uses no external RL or iris library.

```
modal run modal_apps/train.py::run_all --cfg configs/catch_win.yaml --tag win
modal run modal_apps/evals.py::recon   --cfg configs/catch_win.yaml --tag win
modal run modal_apps/train.py::ablate  --cfg configs/catch_win.yaml --tag win
modal run modal_apps/evals.py::eval    --cfg configs/catch_win.yaml --tag win --n 500
modal run modal_apps/evals.py::dreams  --cfg configs/catch_win.yaml --tag win
modal run modal_apps/evals.py::sync
./plotvenv/bin/python make_paper_figures.py
```

The tokenizer fix is two config lines (`ema: true`, `fg_weight: 25`); the ablation in §5 re-runs the tokenizer with each turned off in turn, on the *same* replay buffer, so the before/after comparison is controlled.

8 Conclusion

Building all three IRIS components from scratch and running them on a real GPU budget turns a model-based RL agent into something you can debug end-to-end. The Transformer world model—the piece that sounds hardest—is the easy part: it learns pixel-Catch to 97.8% token accuracy in under a minute of GPU time. The genuinely hard part is upstream, in the discrete representation, and it hides a trap that the usual “codebook collapse” diagnosis walks right past: a background-dominated reconstruction loss silently discards small, fast, task-critical objects, and a perfectly healthy codebook does nothing to stop it. Naming that failure precisely—and fixing it with a one-line foreground-weighted loss on top of standard EMA codebook updates—is what carries the agent from random play (catch rate 0.16) to a policy that learns entirely in imagination (catch rate 1.00, oracle 1.0). **A world model can only teach a policy what its tokenizer chooses to preserve.**

References

- [1] V. Micheli, E. Alonso, and F. Fleuret. Transformers are Sample-Efficient World Models. In *ICLR*, 2023. arXiv:2209.00588.
- [2] D. Ha and J. Schmidhuber. World Models. arXiv:1803.10122, 2018. (NeurIPS 2018, “Recurrent World Models Facilitate Policy Evolution.”)

- [3] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi. Dream to Control: Learning Behaviors by Latent Imagination. In *ICLR*, 2020. arXiv:1912.01603.
- [4] D. Hafner, T. Lillicrap, M. Norouzi, and J. Ba. Mastering Atari with Discrete World Models. In *ICLR*, 2021. arXiv:2010.02193.
- [5] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap. Mastering Diverse Domains through World Models. arXiv:2301.04104, 2023.
- [6] A. van den Oord, O. Vinyals, and K. Kavukcuoglu. Neural Discrete Representation Learning. In *NeurIPS*, 2017. arXiv:1711.00937.
- [7] A. Razavi, A. van den Oord, and O. Vinyals. Generating Diverse High-Fidelity Images with VQ-VAE-2. In *NeurIPS*, 2019. arXiv:1906.00446.
- [8] P. Dhariwal, H. Jun, C. Payne, J. W. Kim, A. Radford, and I. Sutskever. Jukebox: A Generative Model for Music. arXiv:2005.00341, 2020.
- [9] L. Kaiser et al. Model-Based Reinforcement Learning for Atari. In *ICLR*, 2020. arXiv:1903.00374.
- [10] I. Osband et al. Behaviour Suite for Reinforcement Learning. In *ICLR*, 2020. arXiv:1908.03568.